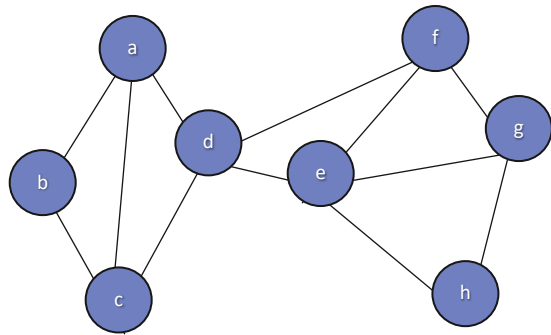


Графовые нейросети и их применение

Обозначения



$G = (V, E)$, где

G — структура, которая состоит из двух множеств

V — множество вершин и их признаков

E — множество ребер и их признаков

$|V|$ - количество вершин

$|E|$ - количество ребер

u, v - вершины

d_u - степень вершины u

e_{uv} - признаки ребра, соединяющие вершины u, v

Ограничения классических GNN

Классическая GNN - это

- **Локальная агрегация соседей (message passing)**
- **Повторение этого процесса L раз**
- **Получение L -hop информации**

Но у этого подхода есть фундаментальные ограничения, которые начали проявляться по мере применения GNN к более сложным задачам и масштабам.

Ограничения классических GNN

Ограничение 1: локальность (k-hop neighborhood)

GNN видит только L-hop окрестность:

- если $L = 2 \rightarrow$ модель видит только соседей соседей,
- если $L = 3 \rightarrow$ только третье поколение соседей.

Проблема:

многие задачи требуют глобального контекста.

Примеры:

- молекулы, где важны long-range взаимодействия атомов
- задачи на графах знаний, где сигнал может идти через 5, 7 или 10 рёбер
- социальные сети, где влияние пользователя распространяется далеко по структуре

Классическая GNN в таких случаях вынуждена увеличивать $L \rightarrow$ и сталкивается со следующей проблемой. ⁴

Ограничения классических GNN

Ограничение 2: over-smoothing

Это ключевой теоретический недостаток message passing:

При большом числе слоёв GNN embeddings всех узлов становятся одинаковыми.

Это доказано как теоретически (их спектральные свойства показывают сглаживание), так и экспериментально.

Эффект:

- сигнал быстро теряет уникальность,
- узлы становятся неразличимы,
- качество резко падает.

Это делает GNN **практически неспособными работать глубже 3-4 слоёв.**

Ограничения классических GNN

Ограничение 3: over-squashing

Это более тонкая, но глубокая проблема.

Когда GNN пытается передать информацию из экспоненциально растущей области графа через узкие bottleneck-слои → информация «сжимается» (“squashed”).

Представьте, что один узел должен собрать сигнал из огромного подграфа:

- размер информации растёт экспоненциально,
- но его embedding фиксированной размерности d .

Следствие:

- GNN **не умеет агрегировать много информации одновременно**,
- длинные пути становятся информационно недоступны.

Transformer этой проблемой **не страдает**, т.к. имеет глобальное внимание.

Ограничения классических GNN

Ограничение 4: ограниченная выразительность (1-WL)

Большинство message-passing GNN имеют такую же силу различения, как **1-WL тест**.

Это означает:

- две разные структуры графа могут быть *нерешаемы* для GNN,
- даже если различия видны человеку,
- многие задачи требуют более мощной структуры.

Graph Transformers потенциально могут иметь **выразительность выше WL** благодаря глобальному вниманию и positional encodings.

Ограничения классических GNN

Ограничение 5: нет учёта сложной глобальной структуры

GNN опирается на локальную структуру (кто соседи), но:

- не знает расстояний между узлами,
- не знает глобальной позиции узла в графе,
- не знает роли узла в макроструктуре (hub, bridge, boundary, authority, community node),
- не учитывает многомерные зависимости между узлами.

Transformers позволяют это исправить через:

- структурные bias в attention,
- positional encodings,
- edge encodings,
- distance encodings.

Почему возникла идея перенести Transformer на графы

Почему возникла идея перенести Transformer на графы

Transformer стал прорывом в NLP, а затем - в CV, multimodal tasks, RL, protein folding.

Ключевые свойства, которые сделали его универсальным:

Свойство 1: глобальный механизм внимания

Каждый токен может напрямую смотреть на любой другой.

На графах это означает:

- узел может учитывать информацию от всех узлов,
- даже если между ними много рёбер,
- без необходимости проходить L-hop пути.

Это резко увеличивает доступность глобального контекста.

Почему возникла идея перенести Transformer на графы

Свойство 2: способность моделировать сложные зависимости

В NLP attention учится «понимать»:

- синтаксис,
- дальние зависимости,
- coreference,
- семантические отношения.

В графах такие зависимости тоже есть:

- структурные роли узлов (hub, gateway),
- short vs long cycles,
- structural equivalence,
- distant but functionally related nodes.

Почему возникла идея перенести Transformer на графы

Свойство 3: универсальность архитектуры

Transformers работают на:

- последовательностях,
- изображениях (ViT),
- таблицах,
- звуке,
- многомерных токенизированных структурах.

Основная идея Graph Transformers

Убрать локальность GNN → позволить узлу смотреть на любой другой узел, но **адаптировать attention под структуру графа**.

Главная формула attention:

$$\alpha_{ij} = \exp \frac{\left(\frac{Q_i^\top K_j}{\sqrt{d}} + b_{ij} \right)}{\sum_k \left(\frac{Q_i^\top K_k}{\sqrt{d}} + b_{ik} \right)}$$

Где:

- b_{ij} - **структурный bias**, модифицирующий attention согласно структуре графа.

Основная идея Graph Transformers

Как адаптируют Transformer к графам

GNN → локальные операции

Transformer → глобальные операции

В Graph Transformers:

1. Узел = токен
2. Граф = множество токенов
3. Self-Attention = глобальное взаимодействие
4. Structural Encoding = «позиционная адаптация» к топологии
5. Edge Encoding = семантика связей
6. Graph Tokens = дополнительные узлы (как [CLS] в NLP)

Основная идея Graph Transformers

Примеры задач, где нужен глобальный контекст

1. Молекулы

Сильные дальние взаимодействия в химии:

- водородные связи
- пространственные взаимодействия

2. Белки

Дальний контакт аминокислот определяет folding.

3. Рекомендации

Два пользователя могут быть связаны через длинные цепочки взаимодействий.

4. Оценка релевантности на KGs

Отношения могут проходить через длинные path-цепочки.

5. Графы в NLP / AMR

Семантические зависимости являются неглубокими, но не локальными.

Классический Self-Attention

Прежде чем адаптировать его для графов, нужно чётко понимать механизм.

Пусть дан набор токенов:

$$X = \{x_1, x_2, \dots, x_n\}$$

Каждый токен преобразуется в Query, Key, Value:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

Затем self-attention вычисляется:

$$Att(Q, K, V) = softmax \left(\frac{QK^T}{\sqrt{d}} \right) V$$

Важные моменты:

1. Взаимодействуют все токены → глобальный контекст.

2. Нет ограничений расстояний.

3. Нет топологии, кроме позиции (positional encoding).

Это работает в NLP (где последовательность линейна), но для графов всё ломается.

Классический Self-Attention НЕ подходит

Проблема 1: В графе нет линейного порядка

Нет индексации 1, 2, 3...

Любая перестановка узлов должна давать **один и тот же результат**.

Transformer в NLP → не инвариантен к перестановкам.

На графе → должен быть **permutation invariant / equivariant**.

Проблема 2: Attention не знает топологии

Классический attention вычисляет:

$$\alpha_{ij} = \frac{Q_i^T K_j}{\sqrt{d}}$$

Но Q и K - *функции только самих узлов*, поэтому без дополнительных сигналов:

Модель не знает, на каком расстоянии находятся узлы, связаны ли они ребром, и какую роль играют в структуре.

Классический Self-Attention НЕ подходит

Проблема 3: Масштабируемость

Self-attention = $O(N^2)$, что при больших графах неэффективно.

Нужны sparse-подграфы, группировка узлов, или эффективные PE.

Проблема 4: Узлы могут быть структурно эквивалентны

Transformer не различает узлы:

- занятые в роли “hub”,
- лежащие на циклах,
- рядом с мостами,
- boundary nodes.

Эта семантика не выражается в embedding X.

Адаптация Self-Attention к графам

Чтобы Self-Attention работал на графах, его нужно модифицировать в двух направлениях:

1. Добавить структурную информацию в attention-матрицу

Через bias:

$$\alpha_{ij} = \frac{Q_i^T K_j}{\sqrt{d}} + b_{ij}$$

Где b_{ij} — структурный сигнал:

- кратчайшее расстояние (SPD)
- число путей между i и j
- типы рёбер между i и j
- спектральный сигнал
- центральность
- роль узла

Эта идея лежит в основе всех современных Graph Transformers (Graphormer, SAN, GPS).

Адаптация Self-Attention к графам

Чтобы Self-Attention работал на графах, его нужно модифицировать в двух направлениях:

2. Определить “позицию” узла не в линейном порядке, а в графовой топологии

Через **graph positional encodings (GPE)**.

PE становятся:

- **структурным описанием узла,**
- **подписью его позиции в графе.**

Определение Graph-Aware Attention

Graph-aware attention - это внимание, модифицированное структурной информацией.

Общая формула:

$$Att(i, j) = softmax \left(\frac{Q_i^T K_j}{\sqrt{d}} + b(\text{graph structure between } i, j) \right) V_j$$

Где $b(..//..)$ - это:

- либо добавка к QK ,
- либо дополнительный embedding,
- либо edge-based bias,
- либо attention mask,
- либо gating-функция (GraphGPS).

Как именно добавляют структуру?

1. Типовые структурные расстояния

Вводят bias по кратчайшим расстояниям:

$$b_{ij} = \text{bias}(\text{SPD}(i, j))$$

Используется в Graphormer.

2. Edge Encodings (типы рёбер и связи)

Если между i и j есть ребро:

$$b_{ij} = e(i, j)W_E$$

или набор релевантных сигналов:

- тип ребра
- вес ребра
- направленность
- количество параллельных связей

Определение Graph-Aware Attention

Node-Node Relative Position Encodings

Определение относительной позиции двух узлов:

- расстояние
- количество простых путей
- длина симплициальных комплексов
- “structural role similarity”
- спектральные признаки

Используют:

- SAN
- GraphGPS
- TokenGT

В Transformer это заменяет abs-positional encoding из NLP.

Определение. Graph Positional Encodings

Графовый positional encoding - это отображение:

$$PE: V \rightarrow \mathbb{R}^d$$

или

$$PE_{rel}: V \times V \rightarrow \mathbb{R}^d$$

такое, что:

1. сохраняет графовую структуру,
2. инвариантно к изоморфизмам:

$$PE(Pv) = P \cdot PE(v) \text{ для любой перестановки } P$$

3. обладает “континуальностью”: небольшое изменение графа $\rightarrow$ небольшое изменение PE.

Графовые метрики как основа PE

Графовые метрики как основа PE

Основные графовые расстояния:

- кратчайший путь
- random walk distance
- commute time
- resistance distance
- diffusion distance

Важное свойство:

любая графовая метрика → потенциальный кандидат для относительных PE

Определение. Graph Positional Encodings

1. Laplacian Eigenvectors (LPE)

Лапласиан:

$$L = D - A$$

Спектральное разложение:

$$L = U\Lambda U^T$$

Берём k-мерный спектральный embedding:

$$PE_i = (U_{i1}, \dots, U_{ik}).$$

Свойства (важные для Graph Transformers):

- спектральное embedding минимизирует energy functional,
- eigenvectors устойчивы к глобальной форме графа,
- eigenvectors различают кластеры через Fiedler vector,
- спектр Лапласиана — изоморфный инвариант (до перестановки).

Почему это PE?

Потому что eigenvectors задают **графовую систему координат**, аналог Fourier-базиса.

Graph Positional Encodings (GPE)

2. Random Walk Structural Encodings (RWSE)

Посчитанное распределение случайного блуждания:

$$PE_i = (p^{(1)}(i), \dots, p^{(k)}(i))$$

Где $p^{(t)}(i)$ - вероятности посещения узлов через t шагов.

Что означает?

- вероятность вернуться в узел через 1,2,...,k шагов
- отражает локальную и средне-дальнюю структуру

Свойства:

- устойчиво к изоморфизмам,
- легко масштабируется: $O(k|E|)$,
- соответствует фундаментальной матрице Марковских процессов.

Graph Positional Encodings (GPE)

3. Shortest Path Distance Encodings (SPD)

Используются как attention bias.

$$d(i, j) = \text{shortestpath}(i, j)$$

Используется в Graphormer как bias:

$$b_{ij} = \text{Emb}(d(i, j))$$

Другие расстояния:

- Resistance distance
- Diffusion distance

Полезно для графов с шумом или manifold-структурой.

Graph Positional Encodings (GPE)

4. Structural/Role Encodings

Формально:

роль узла - орбита его автоморфизмов.

Например:

- степень
- центральность
- кластерный коэффициент
- PageRank
- betweenness

Все они являются инвариантами под перестановками

Graph Positional Encodings (GPE)

5. Relative Position Encodings (RPE)

В Transformer используются *относительные PE*, и это переносится на графы.

Определение:

$$PE_{rel}(i, j) = f(\text{графовых отношений между } i \text{ и } j)$$

Примеры:

- SPD-bias
- Edge-type bias
- RW-distance relational encoding
- Reduction of LPE to pairwise form

В attention:

$$\alpha_{ij} = \text{softmax} \left(\frac{Q_i^\top K_j}{\sqrt{d}} + PE_{rel}(i, j) \right)$$

Это делает attention **структурно осведомлённым**.

Инвариантность и теоретическая выразительность

Глубокий результат:

Если GPE инвариантны к изоморфизмам, а attention equivariant к перестановкам, Graph Transformer является permutation equivariant.

Также:

Graph Transformers с правильными PE **могут превосходить 1-WL тест**.

Особенно SPD + RPE.

Потому что attention с RPE различает относительные позиции узлов.

Graphormer - один из первых примеров этого.

Ограничения и проблемы PE

1. Laplacian PE плохо масштабируется
2. SPD требует $O(N^2)$ расстояний
3. PE может ломать permutation invariance
4. PE могут плохо переноситься между разными графами
5. Графы с большим количеством узлов $\rightarrow$ слишком большой bias matrix

Классификация архитектур Graph Transformers

Почему вообще нужно выделять классы Graph Transformers?

Формула self-attention слишком гибкая.

Она допускает множество вариантов:

- dense attention vs sparse attention,
- расстояние-aware vs purely content-based,
- с использованием MPNN-слоёв или полностью независимые от них,
- с различными PE (Laplacian, SPD, RWSE, role-based, RPE),
- с global tokens, edge tokens, structural biases.

Поэтому существует **десятки** моделей, и нам нужен формальный список классов.

Классификация архитектур Graph Transformers

Класс I: Dense Self-Attention Graph Transformers

(полное $O(N^2)$ взаимодействие между всеми узлами)

Идея:

Все узлы смотрят на все узлы:

$$\alpha_{ij} = \text{softmax} \left(\frac{Q_i K_j^\top}{\sqrt{d}} + b_{ij} \right)$$

Особенности:

- глобальное внимание,
- высокая выразительность,
- дорогие $O(N^2)$ вычисления,
- требует bias-матриц → PE критически важны.

Классификация архитектур Graph Transformers

Плюсы:

- самая мощная архитектура;
- превосходит GNN на задачах OGB-LSC;
- легко расширяется до foundation-моделей.

Минусы:

- $O(N^2)$ память и время;
- не подходит для больших социальных/веб-графов;
- сильная зависимость от PE.

Классификация архитектур Graph Transformers

Класс II: Sparse Graph Transformers (Neighbourhood-based Attention)

(attention только на соседях или k-hop)

Идея:

Attention заменяет или дополняет message-passing:

$$\alpha_{ij} = 0, \text{ если } j \notin N_k(i)$$

Вместо полного внимания → только локальное.

Особенности:

- масштабируемость,
- хороши для больших графов,
- attention остаётся структурно информированным.

Классификация архитектур Graph Transformers

Плюсы:

- работает на больших графах;
- гибкость между GNN и Transformer;
- можно использовать на миллионах узлов.

Минусы:

- теряет часть глобального контекста;
- структурная выразительность ниже класса I.

Классификация архитектур Graph Transformers

Класс III: Hybrid GNN + Transformer (GraphGPS-подход)

(GNN даёт inductive bias, Transformer даёт глобальность)

Идея:

Комбинировать лучшее от двух миров:

$$h^{(l+1)} = MPNN(h^{(l)}) + TransformerLayer(h^{(l)})$$

То есть:

- MPNN → локальный inductive bias (множество паттернов в графах)
- Transformer → глобальный контекст

Особенности:

- устойчивее к шуму, чем чистый Transformer,
- работает на больших графах лучше dense моделей,
- выдаёт SOTA на многих задачах (OGB, молекулы).

Классификация архитектур Graph Transformers

Плюсы:

- масштабируемость;
- высокая точность;
- MPNN компенсирует слабые PE.

Минусы:

- сложность реализации;
- нужно тщательно подбирать PE.

Классификация архитектур Graph Transformers

Класс IV: Pure Token Transformers (Graph as a token set)

(граф → множество токенов, никаких GNN-операций)

Идея:

Трактовать узлы как независимые токены:

- без message-passing,
- без adjacency,
- без GNN-операций.

Структура даётся **только через PE**:

$$\alpha_{ij} = \text{softmax} \left(Q_i^\top K_j + PE_{rel}(i, j) \right)$$

Особенности:

- очень универсальные, как ViT → GraphViT
- потенциально совместимы с огромными моделями (GraphGPT)

Классификация архитектур Graph Transformers

Плюсы:

- архитектурная простота;
- можно обучать очень большие модели;
- радикальная переносимость (как у BERT/Vision Transformers).

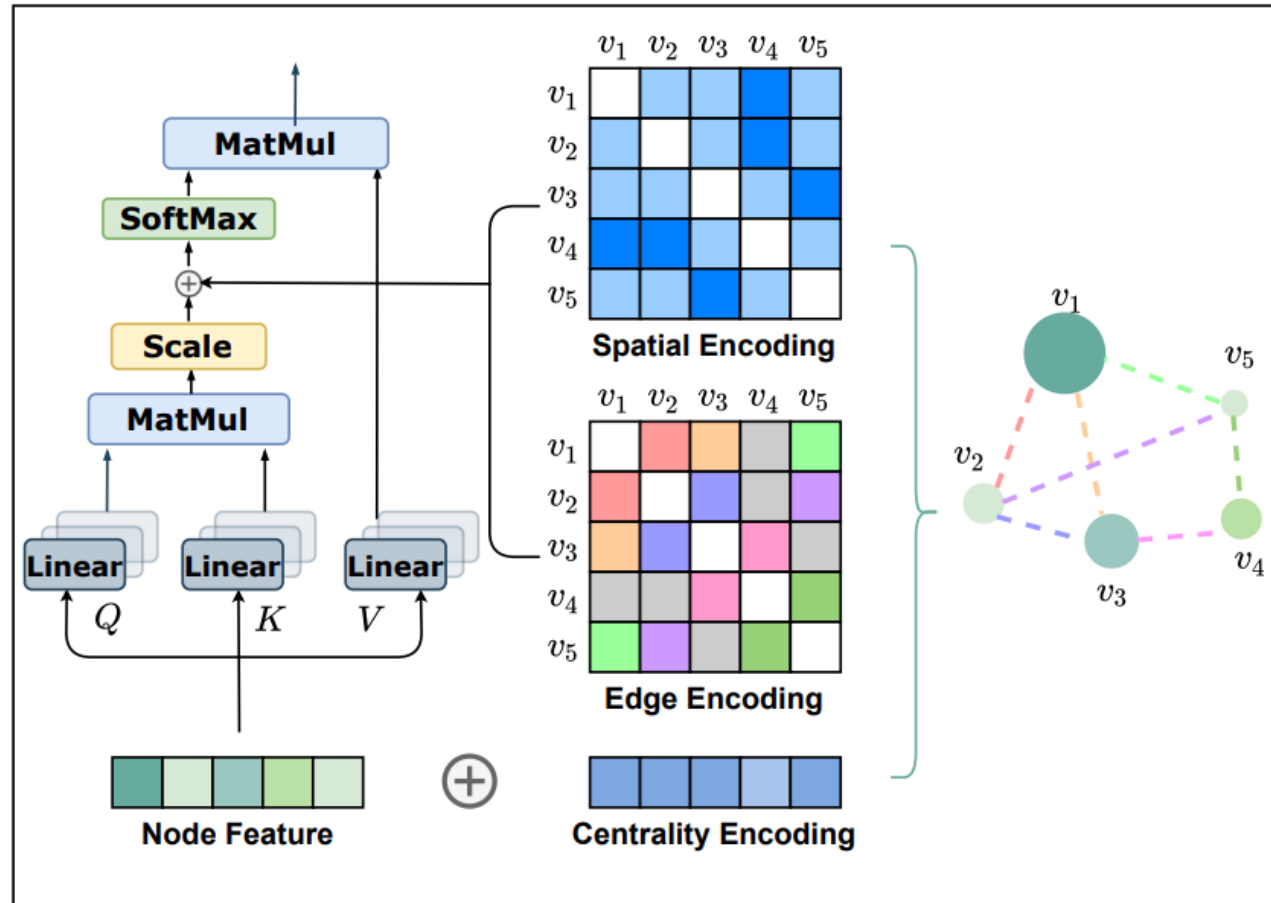
Минусы:

- полностью зависит от качества PE;
- без хороших PE модель теряет структуру;
- дорогой $O(N^2)$ attention.

Классификация архитектур Graph Transformers

Класс	Attention	Масштаб	PE-зависимость	Относительная выразительность
Dense	$O(N^2)$	низкая	высокая	очень высокая
Sparse	локальный	высокая	средняя	сопоставима с GNN+
Hybrid	глобальный+локальный	высокая	средняя	высокая
Token	$O(N^2)$	низкая	крайне высокая	максимальная при хороших PE

Graphormer



Источник: <https://arxiv.org/pdf/2106.05234>

Graphormer

Graphormer берёт классический Transformer и **добавляет графовые structural encodings напрямую в Attention**, а не в токены.

Классический attention:

$$\alpha_{ij} = \frac{Q_i^\top K_j}{\sqrt{d}}$$

Graphormer attention:

$$\alpha_{ij} = \frac{Q_i^\top K_j}{\sqrt{d}} + b_{SPD(i,j)} + b_{edge(i,j)} + c_i + c_j$$

То есть графовые PE = набор структурных bias-ов:

- **SPD bias** - кратчайшие расстояния
- **Edge Encoding** - типы и веса рёбер
- **Centrality Encoding** - степень узла

Все bias складываются в attention → трансформер становится структурно осведомлённым.

Graphormer

Компонент 1: Centrality Encoding

В классическом Transformer токены - слова, у которых есть позиция.

В графе узлы не имеют позиции, но имеют **структурную важность**.

Graphormer вводит **centrality embedding**:

$$h_i^{(0)} = x_i + C(\text{deg}(i))$$

Где C - обучаемый embedding степени.

Зачем?

- степень - инвариант графа,
- она отражает локальную роль узла,
- добавляет важную bias-информацию в Q, K, V.

Это похоже на “первичный positional encoding”.

Graphormer

Компонент 2: Spatial Encoding (SPD bias)

Главный компонент Graphormer. Это именно то, что принесло ему успех.

Определение:

Для каждого узла i и j :

$$d_{ij} = SPD(i, j)$$

И для каждого расстояния есть embedding:

$$b_{ij}^{SPD} = Emb(SPD(i, j))$$

Добавляется в attention:

$$\alpha_{ij} = \frac{Q_i^\top K_j}{\sqrt{d}} + b_{ij}^{SPD}$$

Graphormer

Компонент 3: Edge Encoding (edge-level bias)

Для рёбер (i,j):

$$b_{ij}^{edge} = Emb(E(i,j))$$

Где E(i,j) - структурные/химические признаки ребра:

- тип химической связи (single/double/etc),
- aromatic bond,
- расстояние (в 3D),
- charge transfer,
- в других графах: тип отношения, вес, направление.

Edge encoding - сильный сигнал, особенно в химии.

Он подсказывает attention, через какие рёбра информация должна течь быстрее.

Graphormer

Полная формула attention в Graphormer

Все bias суммируются:

$$\alpha_{ij} = \frac{Q_i^\top K_j}{\sqrt{d}} + b_{SPD(i,j)} + b_{edge(i,j)} + c_i + c_j$$

Затем softmax:

$$a_{ij} = \text{softmax}(\alpha_{ij})$$

Это *структурно-информированный attention*, который учитывает:

- глобальные расстояния,
- локальные связи,
- степени узлов,
- edge features.

Graphormer

Почему Graphormer решил проблему Graph Transformers?

1. Инвариантность к перестановкам

Bias функции зависят от структуры графа, а не от индекса узла → инвариантность соблюдается.

2. Глобальный контекст без exploding paths

SPD bias делает глобальные связи явно заметными.

3. Отсутствие over-smoothing

Нет локального message passing → нет экспоненциального сглаживания.

4. Решение проблемы over-squashing

Attention напрямую соединяет дальние узлы.

5. Отличная обучаемость

Доказано на OGB-LSC:

Graphormer учится лучше и быстрее GNN.

Graphormer

Архитектура Graphormer (детали)

Graphormer использует:

- L Transformer-слоёв
- Multi-head attention
- Feed-forward layers
- Residual connections
- LayerNorm
- Dropout and attention dropout
- Mean pooling для graph-level задач

Никакого:

- MPNN,
- Convolution over neighbors,
- GNN-операторов.

Graphormer — **чистый Transformer**, усиленный структурными bias.

Graphormer

Graphormer стал победителем **OGB-LSC PCQM4Mv2**,
который был крупнейшим соревнованием по ML на графах.

Причины успеха:

- огромная устойчивость к большим графам,
- правильный bias для химии: SPD + edge-type,
- мощная глобальная обработка сигналов,
- минимальная зависимость от количества рёбер.

Другие модели отставали на 20-30%.

Graphormer также стал SOTA на:

- ZINC (molecule optimization),
- QM9 properties,
- Metaphoric prediction tasks.

Ограничения Graphormer

1. $O(N^2)$ время и память

Нужно sparse attention или approximation для больших графов.

2. Требуется $O(N^2)$ SPD bias

Для огромных графов (100k+ узлов) это неэффективно.

3. Не универсально переносим

SPD bias - граф-специфичен.

4. Плохая интерпретируемость на больших графах

Развитие идей Graphormer

Graphormer дал начало целому семейству моделей:

GraphormerV2: sparse-RPE, multi-graph pretraining

GraphGPS: MPNN + Transformer

TokenGT: pure token-based, без SPD

GraphMamba: state space models with graph bias

Все современные Graph Transformers так или иначе используют bias-механизмы Graphormer.

Эволюция

Годы	Эпоха	Основные идеи
2018	Attention в графах	GAT, ограниченное локальным вниманием
2019	Первые Graph Transformers	Graph-BERT, Laplacian PE
2020	Теоретическая база	PE, спектральные методы
2021	Прорыв: Graphormer	SPD, edge bias, OGB-LSC победа
2022	Масштабируемые модели	SAN, GraphGPS
2023	Token-based эпоха	TokenGT (ViT для графов)
2024	Универсальные модели	GraphormerV2, GraphMamba, GraphGPT

Источники по Graph Transformers и их компоненты

Основной источник:

- Stanford CS224W: Machine Learning with Graphs — Основной академический курс по GNN.
<http://snap.stanford.edu/class/cs224w>
- **Graphormer - Do Transformers Really Perform Bad for Graph Representation?**
Ying, C., Cai, T., Luo, S., Zheng, S., Ke, G., He, D., Shen, Y., Liu, T.-Y. (2021).
<https://arxiv.org/abs/2106.05234>
- **Официальный GitHub Graphormer (Microsoft)**
<https://github.com/microsoft/Graphormer>
- **GraphGPS - General, Powerful, Scalable Graph Transformer**
Rampášek, L., Galkin, M., Dwivedi, V. P., Luu, A. T., Wolf, G., Beaini, D. (2022). *Recipe for a General, Powerful, Scalable Graph Transformer (GraphGPS)*.
<https://arxiv.org/abs/2205.12454>
- **Официальный GitHub GraphGPS**
<https://github.com/rampasek/GraphGPS>
- **Attending to Graph Transformers (ранний обзор)**
Müller, L., Dwivedi, V. P., Rampášek, L., Beaini, D. (2023). *Attending to Graph Transformers*. TMLR.
<https://openreview.net/forum?id=HhbqHBBrfZ>

Источники по Graph Transformers и их компоненты

Основной источник:

- **Graph Transformers: A Survey**
Shehzad, A., et al. (2024). *Graph Transformers: A Survey*. arXiv.
<https://arxiv.org/abs/2407.09777>
- **A Survey of Graph Transformers: Architectures, Theories and Applications**
Yuan, C., Zhao, K., et al. (2025). *A Survey of Graph Transformers: Architectures, Theories and Applications*. arXiv.
<https://arxiv.org/abs/2502.16533>
- **PyTorch Geometric - Graph Transformer Tutorial**
- Практический tutorial по реализации Graph Transformer/GraphGPS в PyTorch Geometric: пример кода, настройка positional encodings и глобального внимания.
https://pytorch-geometric.readthedocs.io/en/latest/tutorial/graph_transformer.html